Service Blueprint – Two-Tier Web Service

1. Service Overview:

The service is a small two-tier web application. Users send requests through a web interface. The application server processes requests and communicates with a database to store or retrieve information.

2. Architecture:

User → HTTPS Request → Web/Application Server → Database

The application server validates incoming requests, applies business logic and returns responses to users. The database stores application records. Monitoring and logging track errors, latency and availability.

3. Request Flow:

1. The user sends a request over HTTPS.
2. The application validates the request.
3. The application server processes the request.
4. The server reads or writes data in the database.
5. The application returns a response to the user.
6. Logs and metrics record request latency, errors and system health.

4. Dependencies:

- Web hosting or application compute
- Managed database
- Network and DNS
- Authentication and secrets management
- Monitoring and centralized logging
- Backup and recovery services

5. Deployment Flow:

1. Developer commits and reviews code.
2. Automated tests run.
3. The application is built and deployed to a staging environment.
4. Health checks and smoke tests are performed.
5. The release is deployed to production.
6. Monitoring verifies service health; failed releases can be rolled back.

6. Failure Paths:

- Application failure: restart the service and use a previous healthy release if necessary.
- Database failure: retry safe operations, alert the team and restore service using the recovery plan.
- Network failure: return a clear error and retry only when appropriate.
- High traffic: apply rate limiting and scale application instances.
- Invalid requests: return suitable HTTP error codes and log relevant details without exposing sensitive information.

7. Monitoring:

Track availability, p95 request latency, error rate, resource utilization and database health. Alerts should trigger when service objectives are at risk.

8. Security:

Use HTTPS, least-privilege access, managed secrets, restricted database access, encrypted backups and appropriate log retention.